

AEP Standard v0.1

AI Engineering Package Protocol Draft / AI 工程包协议标准草案

Raphael Davad & ChatGPT

2026-06-02

目录

AEP Standard v0.1	2
0. 摘要	2
1. 背景: 为什么需要 AEP	3
2. AEP 的定位	3
3. AEP 与 ZIP / RAR 的区别	4
4. AEP 的六个核心原则	4
4.1 自包含	4
4.2 可迁移	5
4.3 AI 可读	5
4.4 可执行	5
4.5 可验证	5
4.6 可恢复	5
5. AEP 的核心概念	6
5.1 工程工作单元	6
5.2 任务外包协议	6
5.3 工程工作量证明 PoEW	6
6. AEP v0.1 标准目录结构	7
7. 核心文件职责	8
7.1 00_START_HERE.md	8
7.2 manifest.json	8
7.3 human_brief.md 和 human_brief.pdf	8
7.4 agent_spec.md	9
7.5 rules.md	9
7.6 task_map.yaml	9
7.7 tasks/	10
7.8 execution/	10
7.9 validation/	11
7.10 outputs/	11
7.11 reports/	11
7.12 changelog.md	11
8. 标准执行流程	11
9. 状态机	12
9.1 AEP 包状态	12
9.2 任务状态	12

10. 验收模型	12
10.1 PASS	13
10.2 HOLD	13
10.3 FAIL	13
10.4 PARTIAL	13
10.5 ADOPT	13
11. validation_rules.yaml 示例	13
12. 任务卡模板	14
13. AEP 的最小可用版本	15
14. AEP 与 GitHub 项目的关系	16
15. AEP 未来路线图	16
v0.1: 协议草案	16
v0.2: 模板包	17
v0.3: Validator	17
v0.4: Runner	17
v0.5: GitHub 项目化	17
v1.0: 协议稳定版	17
16. AEP 的研究问题	18
16.1 任务语义标准化	18
16.2 AI 可读性	18
16.3 验收标准设计	18
16.4 工作量证明	18
16.5 安全边界	18
16.6 任务颗粒度	18
16.7 交接机制	18
17. 第一批应用场景	18
18. AEP 的哲学定位	19
19. 最终定义	19
20. 附录: AEP v0.1 最小文件清单	20
21. 附录: AEP v0.1 入口文件模板	20
22. 附录: AEP v0.1 GitHub README 摘要	21

AEP Standard v0.1

全称: AI Engineering Package Standard
中文名: AI 工程包协议标准
缩写: AEP
版本: v0.1 Draft
定位: 面向 AI 协作时代的标准化工程工作单元协议
建议实现方式: ZIP + Markdown + YAML + JSON + PDF + Reports
第一应用场景: Moodify 主线工程任务包
长期方向: 独立 GitHub 开源项目与 AI 任务外包基础协议

0. 摘要

AEP 是一种面向 AI 协作时代的标准化工程工作单元格式。

它将任务目标、执行规则、输入材料、环境要求、任务地图、验收标准、输出路径、停止条件和最终报告封装在一个可迁移文件中。

任何人、任何 AI、任何计算环境在打开 AEP 后，都应能理解任务、执行任务、验证结果，并生成可追踪的工程工作量证明。

AEP 不试图在第一阶段重新发明压缩算法。它首先是一种工程协议，而不是一种底层压缩格式。

最小实现可以是：

AEP = ZIP + 标准目录结构 + manifest.json + agent_spec.md
+ task_map.yaml + validation_rules.yaml + reports

因此，AEP 的第一阶段目标不是取代 ZIP，而是在 ZIP 之上定义 AI 时代的工程任务封装标准。

1. 背景: 为什么需要 AEP

AI 时代的工程协作正在从“人写需求, 人执行”转向“人定义意图, AI 执行工程做功”。

但是，目前大多数 AI 协作仍然停留在聊天窗口中：

用户口头描述需求
AI 临时理解上下文
AI 给出一次性结果
结果散落在聊天记录、代码目录和本地文件中
验收标准模糊
失败难以复盘
下一个 AI 难以接手

这种方式适合一次性问答, 但不适合长期工程。

长期工程需要的是：

任务可封装
规则可读取
边界可限制
输入可追踪
输出可验证
失败可分类
结果可复盘
工作可交接

AEP 正是为了解决这个问题。

它的核心命题是：

如何把人的工程意图，封装成 AI 可读取、可执行、可验证、可交接的标准工作单元？

2. AEP 的定位

AEP 不是一个普通压缩包。

AEP 是一种工程语义协议。

它和 ZIP 的关系是:

ZIP 是物理容器。

AEP 是工程协议。

.aep 可以是未来的标准后缀。

更准确地说:

AEP v0.1 = 一个遵守 AEP 协议的 ZIP 工程包。

未来当协议稳定后, .aep 可以成为一种专门后缀。其内部仍然可以采用 ZIP 结构, 就像 .docx、.xlsx、.epub 等格式本质上也使用 ZIP 作为容器一样。

3. AEP 与 ZIP / RAR 的区别

项目	ZIP / RAR	AEP
本质	压缩格式	工程协议
目标	打包和传输文件	封装可执行工程任务
是否规定目录结构	否	是
是否规定 AI 读取入口	否	是
是否包含任务地图	不要求	必须包含
是否包含验收标准	不要求	必须包含
是否包含停止条件	不要求	必须包含
是否包含输出路径	不要求	必须包含
是否支持工作量证明	不支持	支持
是否适合 AI 外包	只适合作为容器	适合作为协议单元

一句话:

ZIP 只是把文件装起来。

AEP 规定文件如何组织、AI 如何执行、结果如何验收。

4. AEP 的六个核心原则

每一个 AEP 工程包必须尽可能满足六个原则:

Self-contained	自包含
Portable	可迁移
Agent-readable	AI 可读
Executable	可执行
Verifiable	可验证
Resumable	可恢复

4.1 自包含

AEP 包内部必须包含执行该任务所需的关键说明、任务边界、输入描述、输出要求、验收标准和报告模板。

执行者不应严重依赖外部聊天上下文。

4.2 可迁移

AEP 包应可以被复制到任意电脑、云服务器、CI 环境或 AI Agent 工作目录中。

解压后, 执行者应能通过 00_START_HERE.md 开始任务。

4.3 AI 可读

AEP 必须区分人类阅读文件和 AI 执行文件。

例如:

human_brief.pdf	给人看
agent_spec.md	给 AI 看
manifest.json	给机器看
task_map.yaml	给任务调度器看

4.4 可执行

AEP 不应只是理念说明。它必须包含明确的执行路径:

- 先读什么
- 再做什么
- 允许改什么
- 禁止改什么
- 输入在哪里
- 输出到哪里
- 运行什么命令
- 什么时候停止

4.5 可验证

AEP 必须包含验收规则。

完成不能只靠“感觉差不多”, 而要能回答:

- 是否生成了要求文件?
- 是否通过了测试?
- 是否有日志?
- 是否有报告?
- 是否满足 Gate 条件?
- 是否允许进入下游任务?

4.6 可恢复

AEP 应支持中断恢复。

长任务失败或中断后, 新的 AI 或执行者应能通过日志、任务状态和 resume protocol 判断:

哪些已经完成?
哪些失败了?
失败原因是什么?
是否可以重试?
从哪里继续?

5. AEP 的核心概念

5.1 工程工作单元

AEP 的最小封装对象不是“想法”，而是一个可验收的工程工作单元。

它可以是：

- 一个软件节点
- 一个实验任务
- 一个数据清洗任务
- 一个模型评测任务
- 一个论文复现实验
- 一个音频处理批任务
- 一个专利草案任务
- 一个商业分析任务
- 一个内容生产任务

5.2 任务外包协议

AEP 可以被理解为 AI 时代的任务外包基础协议。

传统任务外包依赖会议、聊天、合同和人工理解。

AEP 试图将外包任务结构化为：

- 目标
- 规则
- 输入
- 边界
- 执行
- 验收
- 证明
- 交接

5.3 工程工作量证明 PoEW

AEP 的最终产物不只是结果文件，还应包含工程工作量证明。

建议命名：

PoEW = Proof of Engineering Work
工程工作量证明

PoEW 至少应回答：

执行了哪些任务?
使用了哪些输入?
运行了哪些命令?
修改了哪些文件?
生成了哪些输出?
遇到了哪些失败?
如何分类失败?
是否满足验收标准?
是否允许进入下一阶段?

6. AEP v0.1 标准目录结构

AEP v0.1 推荐采用如下最小目录结构:

AEP_PACKAGE_NAME_AEP_v0.1/

```
|—— 00_START_HERE.md
|—— manifest.json
|—— human_brief.md
|—— human_brief.pdf
|—— agent_spec.md
|—— rules.md
|—— task_map.yaml
|—— tasks/
|   |—— task_001.md
|   |—— task_002.md
|   |—— gate_001.md
|—— inputs/
|   |—— README.md
|   |—— input_files/
|—— execution/
|   |—— runbook.md
|   |—— commands.md
|   |—— agent_prompt.md
|   |—— resume_protocol.md
|—— validation/
|   |—— acceptance_criteria.md
|   |—— validation_rules.yaml
|   |—— result_schema.json
|—— outputs/
|   |—— README.md
|—— reports/
|   |—— final_report_template.md
|   |—— validation_report_template.md
|—— changelog.md
```

v0.1 阶段不要求每个目录都复杂实现, 但建议保留目录结构, 以便后续扩展。

7. 核心文件职责

7.1 00_START_HERE.md

入口文件。任何人或 AI 打开 AEP 后必须首先阅读。

必须说明:

这是哪个工程包
当前版本是什么
先读哪些文件
按什么顺序执行
禁止做什么
完成后输出什么
最终报告写在哪里

7.2 manifest.json

机器可读清单, 类似 AEP 的文件头。

建议字段:

```
{  
  "format": "AEP",  
  "format_version": "0.1",  
  "package_id": "AEP-EXAMPLE-001",  
  "package_name": "Example Engineering Package",  
  "status": "READY",  
  "created_at": "2026-06-02",  
  "updated_at": "2026-06-02",  
  "entry_file": "00_START_HERE.md",  
  "human_brief": "human_brief.pdf",  
  "agent_spec": "agent_spec.md",  
  "task_map": "task_map.yaml",  
  "rules": "rules.md",  
  "validation": "validation/validation_rules.yaml",  
  "output_dir": "outputs/",  
  "report_dir": "reports/"  
}
```

7.3 human_brief.md 和 human_brief.pdf

人类阅读文件。

用于说明:

为什么做这个任务
它处于什么战略位置
完成后带来什么能力
为什么现在要做
哪些人应该关注它

PDF 用于人类阅读, 不作为 AI 执行依据。

7.4 agent_spec.md

AI 执行主协议。

必须精确说明:

任务定义
本次做什么
本次不做什么
允许修改范围
禁止修改范围
输入路径
输出路径
执行顺序
成功标准
失败标准
停止条件
下游交接

7.5 rules.md

执行红线文件。

必须明确规定:

禁止删除原始数据
禁止擅自扩大任务范围
禁止无日志运行
禁止无限重试
禁止跳过验收
禁止修改未授权模块
禁止隐藏失败
失败必须分类
结果必须写入指定目录

7.6 task_map.yaml

任务地图文件。

用于描述任务之间的依赖关系、优先级和状态。

示例:

```
package_id: AEP-EXAMPLE-001
node_name: Example Engineering Package
status: READY
```

```
tasks:
- id: AEP-T001
  name: Check Environment
  type: check
  priority: P0
  status: TODO
```

```

depends_on: []
task_card: tasks/task_001.md

- id: AEP-T002
  name: Run Baseline Task
  type: run
  priority: P0
  status: TODO
  depends_on:
    - AEP-T001
  task_card: tasks/task_002.md

- id: AEP-GATE001
  name: Acceptance Gate
  type: gate
  priority: P0
  status: TODO
  depends_on:
    - AEP-T002
  task_card: tasks/gate_001.md

```

7.7 tasks/

具体任务卡目录。

每张任务卡只描述一个任务。

任务卡必须包含:

任务编号
 任务名称
 任务目标
 本次只做什么
 本次不做什么
 输入
 输出
 执行步骤
 成功标准
 失败标准
 停止条件
 最终交付物

7.8 execution/

执行目录。

用于保存:

运行手册
 命令模板
 AI prompt

恢复协议
进度检查命令

7.9 validation/

验收目录。

用于保存:

验收标准
验证规则
预期输出
Gate 检查清单
结果 schema

7.10 outputs/

实际输出目录。

所有任务执行结果必须进入 outputs/ 或其中指定子目录。

7.11 reports/

报告目录。

用于保存:

进度报告
最终报告
验证报告
ADOPT / HOLD 决策报告

7.12 changelog.md

变更记录。

记录工程包版本变化、任务调整、状态变更和协议修订。

8. 标准执行流程

任何执行者打开 AEP 后, 推荐按以下顺序执行:

1. 阅读 00_START_HERE.md
2. 读取 manifest.json
3. 阅读 agent_spec.md
4. 阅读 rules.md
5. 读取 task_map.yaml
6. 找到当前任务卡
7. 阅读 validation/acceptance_criteria.md
8. 执行任务
9. 写入 outputs/

10. 生成 reports/
 11. 更新 changelog.md
 12. 给出 PASS / HOLD / FAIL / ADOPT 判断
- 执行者不得跳过 rules.md 和 validation/。

9. 状态机

9.1 AEP 包状态

AEP 包状态只能使用以下值:

状态	含义
DRAFT	草案设计中
READY	可执行
ACTIVE	正在执行
HOLD	暂停, 不进入下游
PARTIAL	部分完成
PASS	当前验收通过
FAIL	失败, 需要重构或重新定义
ADOPT	已采纳, 可作为下游基础
ARCHIVED	已归档

9.2 任务状态

任务状态只能使用以下值:

状态	含义
TODO	未开始
READY	条件已具备
RUNNING	正在执行
PASS	任务通过
FAIL	任务失败
BLOCKED	被依赖阻塞
SKIPPED	本阶段跳过
RETRY	允许重试
DONE	已完成并归档

10. 验收模型

AEP 的验收不应只判断“有没有结果”, 而应判断:

结果是否存在
过程是否可追踪
失败是否被解释

输出是否可复用
是否满足下游需要
是否允许进入 ADOPT

10.1 PASS

当前任务或当前阶段满足既定验收标准。

10.2 HOLD

结果有价值, 但存在关键缺口, 暂时不能作为下游基础。

10.3 FAIL

任务定义、执行过程或结果存在根本问题, 需要重新设计或重跑。

10.4 PARTIAL

部分结果可用, 但不能整体通过。

10.5 ADOPT

结果已通过关键验收, 可以成为后续节点或下游任务的正式基础。

11. validation_rules.yaml 示例

```
validation:
  required_outputs:
    - outputs/result_summary.json
    - reports/final_report.md
    - reports/validation_report.md

  pass_conditions:
    logs_exist: true
    final_report_exists: true
    validation_report_exists: true
    critical_errors: 0

  hold_conditions:
    missing_noncritical_outputs: true
    incomplete_metrics: true
    unresolved_risk_exists: true

  fail_conditions:
    missing_required_outputs: true
    unsafe_operation_detected: true
    repeated_unclassified_error: true
```

```
task_scope_violation: true

adopt_conditions:
  pass_conditions_met: true
  downstream_handoff_ready: true
  reproducibility_notes_exist: true
  known_risks_documented: true
```

12. 任务卡模板

每个任务卡建议使用以下格式:

```
# AEP-T001 Task Name
```

```
## 1. Task Metadata
```

- Task ID:
- Package ID:
- Type:
- Priority:
- Status:
- Depends on:

```
## 2. Task Goal
```

Describe what this task must accomplish.

```
## 3. Scope
```

```
### This task does
```

-
-

```
### This task does not
```

-
-

```
## 4. Inputs
```

-
-

```
## 5. Outputs
```

-
-

6. Execution Steps

- 1.
- 2.
- 3.

7. Success Criteria

-
-

8. Failure Criteria

-
-

9. Stop Conditions

-
-

10. Final Deliverables

- -
-

13. AEP 的最小可用版本

AEP v0.1 的最小可用版本可以只包含以下文件:

00_START_HERE.md
manifest.json
agent_spec.md
rules.md
task_map.yaml
tasks/task_001.md
validation/acceptance_criteria.md
validation/validation_rules.yaml
outputs/
reports/
changelog.md

在这一阶段, human_brief.pdf 可以作为增强文件, 但不是最小必要条件。

14. AEP 与 GitHub 项目的关系

AEP 可以作为一个独立 GitHub 项目存在。

建议仓库名:

aep-standard
ai-engineering-package
aep-protocol

推荐仓库结构:

aep-standard/

```
|—— README.md
|—— LICENSE
|—— docs/
|   |—— AEP_Standard_v0.1.md
|   |—— AEP_Standard_v0.1.pdf
|—— schema/
|   |—— manifest.schema.json
|   |—— task_map.schema.yaml
|   |—— validation_rules.schema.yaml
|—— templates/
|   |—— 00_START_HERE.md
|   |—— agent_spec.md
|   |—— rules.md
|   |—— task_card.md
|   |—— validation_rules.yaml
|—— examples/
|   |—— minimal_aep_package/
|   |—— moodify_mt001_aep/
|—— tools/
|   |—— aep_validate.py
|   |—— aep_pack.py
|—— CHANGELOG.md
```

AEP 项目的长期价值可能不低于某一个具体产品, 因为它试图解决 AI 时代更底层的问题:

任务如何被封装

AI 如何接单

工程如何验收

结果如何证明

工作如何交接

如果 Moodify 是一个 AI 音乐工程系统, 那么 AEP 是更上层的 AI 工程协作基础协议。

15. AEP 未来路线图

v0.1: 协议草案

目标:

定义 AEP 是什么
定义标准目录结构
定义核心文件职责
定义状态机
定义验收模型

v0.2: 模板包

目标:

制作可复制模板
制作 minimal example
制作 Moodify MT-001 示例包

v0.3: Validator

目标:

开发 Python 校验工具
检查目录是否完整
检查 manifest 是否有效
检查 task_map 是否可解析
检查 validation_rules 是否存在

v0.4: Runner

目标:

读取任务地图
辅助执行任务
收集输出
生成 result_summary.json
生成 validation_report.md

v0.5: GitHub 项目化

目标:

建立独立仓库
发布文档
发布模板
发布示例
发布初版 CLI

v1.0: 协议稳定版

目标:

形成稳定 schema
形成跨项目示例
支持 AI Agent 读取

支持工程外包场景
支持团队协作场景

16. AEP 的研究问题

AEP 作为协议, 至少需要持续研究以下问题:

16.1 任务语义标准化

如何把模糊需求转换为可执行任务?

16.2 AI 可读性

如何让不同模型稳定理解同一个任务包?

16.3 验收标准设计

如何定义“完成”, 避免假完成?

16.4 工作量证明

如何证明 AI 或执行者确实做了工程工作?

16.5 安全边界

如何限制 AI 的修改范围、命令权限和数据访问?

16.6 任务颗粒度

一个 AEP 应该封装一个任务、一个节点、一个实验, 还是一个阶段?

16.7 交接机制

如何让另一个 AI 在没有聊天上下文的情况下接手工作?

17. 第一批应用场景

AEP 可以优先用于以下场景:

- AI 软件开发任务包
- AI 科学实验任务包
- AI 数据处理任务包
- AI 论文复现任务包
- AI 音频处理任务包
- AI 项目管理任务包
- AI 专利草案任务包

AI 商业分析任务包
AI 内容生产任务包
AI 模型评测任务包

Moodify 的 Runtime 节点可以成为第一个真实案例:

MT-001_Runtime_Cloud_System_AEP_v0.1.zip

18. AEP 的哲学定位

AEP 的本质不是文件管理, 而是 AI 时代的工程秩序。

过去, 人类通过合同、会议、文档、项目管理工具来协调复杂工作。

AI 时代, 大量任务将由 AI、人类和计算环境共同完成。

这需要一种新的单位:

它像文档, 但不仅给人读。

它像压缩包, 但不仅装文件。

它像工单, 但包含输入、规则、验收和报告。

它像 Docker, 但封装的不是运行环境, 而是工程意图。

它像区块, 但证明的不是算力, 而是工程工作量。

AEP 试图成为这种新单位。

19. 最终定义

AEP 是一种面向 AI 协作时代的标准化工程工作单元协议。

它以现有 ZIP、Markdown、YAML、JSON、PDF 等技术为基础, 通过标准目录结构和执行协议, 将工程任务封装为可迁移、可执行、可验证、可恢复、可交接的工作包。

AEP 的目标不是替代现有文件格式, 而是在现有文件格式之上, 建立 AI 时代的任务外包基础协议。

最终, 一个合格的 AEP 应该让任何人、任何 AI、任何计算环境在打开后都能回答:

我是谁?

我要做什么?

我不能做什么?

输入在哪里?

输出到哪里?

怎么执行?

何时停止?

怎样验收?

如何证明我完成了工程工作?

如何交给下一个执行者?

这就是 AEP Standard v0.1 的核心。

20. 附录: AEP v0.1 最小文件清单

Required:

- 00_START_HERE.md
- manifest.json
- agent_spec.md
- rules.md
- task_map.yaml
- tasks/
- validation/acceptance_criteria.md
- validation/validation_rules.yaml
- outputs/
- reports/
- changelog.md

Recommended:

- human_brief.md
 - human_brief.pdf
 - execution/runbook.md
 - execution/commands.md
 - execution/agent_prompt.md
 - execution/resume_protocol.md
 - validation/result_schema.json
 - reports/final_report_template.md
 - reports/validation_report_template.md
-

21. 附录: AEP v0.1 入口文件模板

START HERE

You are opening an AEP package.

Package

- Package ID:
- Package Name:
- Version:
- Status:

Read Order

1. manifest.json
2. agent_spec.md
3. rules.md
4. task_map.yaml
5. Current task card

6. validation/acceptance_criteria.md

Execution Rule

Do not execute any task before reading rules.md and validation rules.

Output Rule

All generated files must be written into outputs/ or reports/.

Final Response

At the end, report:

- Completed tasks
 - Modified files
 - Generated outputs
 - Logs
 - Validation result
 - PASS / HOLD / FAIL / ADOPT decision
-

22. 附录: AEP v0.1 GitHub README 摘要

AEP Standard

AEP, short for AI Engineering Package, is a protocol for packaging engineering tasks into portable, agent-readable, executable and verifiable work units.

AEP is not a new compression algorithm. It is a protocol built on top of existing formats such as ZIP, Markdown, YAML, JSON and PDF.

The goal of AEP is to make AI-era task outsourcing structured, traceable, verifiable and resumable.